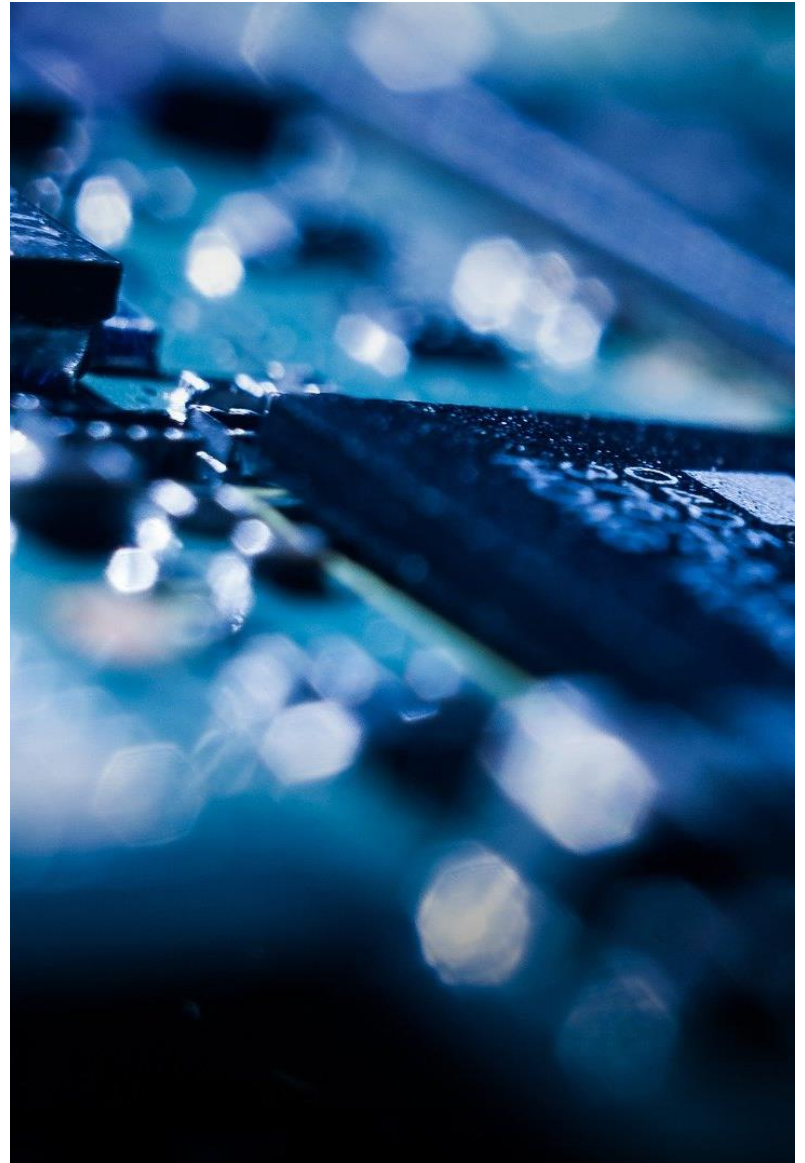


Programmierung eingebetteter Systeme

Pointer und Funktionen

Prof. Dr. Henning Trsek

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



Überblick – Pointer und Funktionen

- ▶ Pointer (Zeiger)
- ▶ Funktionen (Unterprogramme)
- ▶ Variablen
- ▶ Bibliotheksfunktion scanf

Pointer-Variablen (Zeiger)

Ein Zeiger ist eine Variable, die die Adresse eines im Speicher vorhandenen Objekts aufnehmen kann. Eine Zeiger-Variable verweist auf die jeweilige Adresse.



Beispiel:

```
char* xyz_ptr;
```

```
char pe;
```

```
xyz_ptr = &pe;
```

Pointerdefinition:

Pointer auf char-Objekt
(2 Byte) (1 Byte)

Zuweisung: Adresse von pe → Pointer.
Danach zeigt xyz_ptr auf pe.

Zeiger (Beispiel 1)

```
long int* zahl_ptr;
long int  zahl;
long int  a;
```

```
zahl = 0x12345678;
zahl_ptr = &zahl;
a = 0;
```

Zuweisung

```
a = zahl;
a = (long int)&zahl;
a = (long int)zahl_ptr;
a = (long int)&zahl_ptr;
a = *zahl_ptr;
```

Wert von a

```
0x12345678;
0x20000100;
0x20000100;
0x20000000;
0x12345678;
```

Zeiger-Variable
zahl_ptr

...
0x20000000
0x20000001
0x20000002
0x20000003
...

Integer-Variable
zahl

...
0x20000100
0x20000101
0x20000102
0x20000103
...

Integer-Variable
a

...
0x20000200
0x20000201
0x20000202
0x20000203
...

Speicher

Zeiger (Beispiel 1)

```
long int* zahl_ptr;
long int  zahl;
long int  a;
```

```
zahl = 0x12345678;
zahl_ptr = &zahl;
a = 0; ←
```

Zuweisung

```
a = zahl;
a = (long int)&zahl;
a = (long int)zahl_ptr;
a = (long int)&zahl_ptr;
a = *zahl_ptr;
```

Wert von a

```
0x12345678;
0x20000100;
0x20000100;
0x20000000;
0x12345678;
```

Zeiger-Variable zahl_ptr

```
...
0x20000000
0x20000001
0x20000002
0x20000003
```

Integer-Variable zahl

```
...
0x20000100
0x20000101
0x20000102
0x20000103
```

Integer-Variable a

```
...
0x20000200
0x20000201
0x20000202
0x20000203
```

Speicher

Zeiger (Beispiel 1)

```
long int* zahl_ptr;
long int  zahl;
long int  a;
```

```
zahl = 0x12345678;
zahl_ptr = &zahl;
a = 0;
```

Zuweisung

```
a = zahl;
a = (long int)&zahl;
a = (long int)zahl_ptr;
a = (long int)&zahl_ptr;
a = *zahl_ptr;
```

Wert von a

```
0x12345678;
0x20000100;
0x20000100;
0x20000000;
0x12345678;
```

Zeiger-Variable
zahl_ptr

...	...
0x20000000	20
0x20000001	00
0x20000002	01
0x20000003	00

Integer-Variable
zahl

...	...
0x20000100	12
0x20000101	34
0x20000102	56
0x20000103	78

Integer-Variable
a

...	...
...	...
0x20000200	12
0x20000201	34
0x20000202	56
0x20000203	78

Speicher

Zeiger (Beispiel 1)

```
long int* zahl_ptr;
long int  zahl;
long int  a;
```

```
zahl = 0x12345678;
zahl_ptr = &zahl;
a = 0;
```

Zuweisung

```
a = zahl;
a = (long int)&zahl;
a = (long int)zahl_ptr;
a = (long int)&zahl_ptr;
a = *zahl_ptr;
```

Wert von a

```
0x12345678;
0x20000100;
0x20000100;
0x20000000;
0x12345678;
```

Zeiger-Variable
zahl_ptr

...
0x20000000
0x20000001
0x20000002
0x20000003
...

...
20
00
01
00
...

Integer-Variable
zahl

...
0x20000100
0x20000101
0x20000102
0x20000103
...

...
12
34
56
78
...

Integer-Variable
a

...
0x20000200
0x20000201
0x20000202
0x20000203
...

...
20
00
01
00
...

Speicher

Zeiger (Beispiel 1)

```
long int* zahl_ptr;
long int  zahl;
long int  a;
```

```
zahl = 0x12345678;
zahl_ptr = &zahl;
a = 0;
```

Zuweisung

```
a = zahl;
a = (long int)&zahl;
a = (long int)zahl_ptr;
a = (long int)&zahl_ptr;
a = *zahl_ptr;
```

Wert von a

```
0x12345678;
0x20000100;
0x20000100;
0x20000000;
0x12345678;
```

Zeiger-Variable
zahl_ptr

...	...
0x20000000	20
0x20000001	00
0x20000002	01
0x20000003	00

Integer-Variable
zahl

...	...
0x20000100	12
0x20000101	34
0x20000102	56
0x20000103	78

Integer-Variable
a

...	...
0x20000200	20
0x20000201	00
0x20000202	01
0x20000203	00
...	...

Speicher

Zeiger (Beispiel 1)

```
long int* zahl_ptr;
long int  zahl;
long int  a;
```

```
zahl = 0x12345678;
zahl_ptr = &zahl;
a = 0;
```

Zuweisung

```
a = zahl;
a = (long int)&zahl;
a = (long int)zahl_ptr;
a = (long int)&zahl_ptr;
a = *zahl_ptr;
```

Wert von a

```
0x12345678;
0x20000100;
0x20000100;
0x20000000;
0x12345678;
```

Zeiger-Variable
zahl_ptr

0x20000000

0x20000001

0x20000002

0x20000003

...

Integer-Variable
zahl

0x20000100

0x20000101

0x20000102

0x20000103

...

Integer-Variable
a

0x20000200

0x20000201

0x20000202

0x20000203

...

...	...
20	
00	
01	
00	
...	
12	
34	
56	
78	
...	
20	
00	
00	
00	
...	

Speicher

Zeiger (Beispiel 1)

```
long int* zahl_ptr;  
long int  zahl;  
long int  a;
```

```
zahl = 0x12345678;  
zahl_ptr = &zahl;  
a = 0;
```

Zuweisung

```
a = zahl;  
a = (long int)&zahl;  
a = (long int)zahl_ptr;  
a = (long int)&zahl_ptr;  
a = *zahl_ptr;
```

Wert von a

```
0x12345678;  
0x20000100;  
0x20000100;  
0x20000000;  
0x12345678;
```

Zeiger-Variable
zahl_ptr

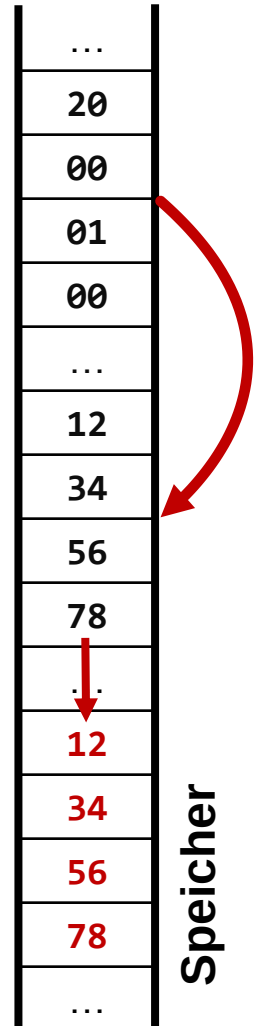
...
0x20000000
0x20000001
0x20000002
0x20000003
...

Integer-Variable
zahl

...
0x20000100
0x20000101
0x20000102
0x20000103
...

Integer-Variable
a

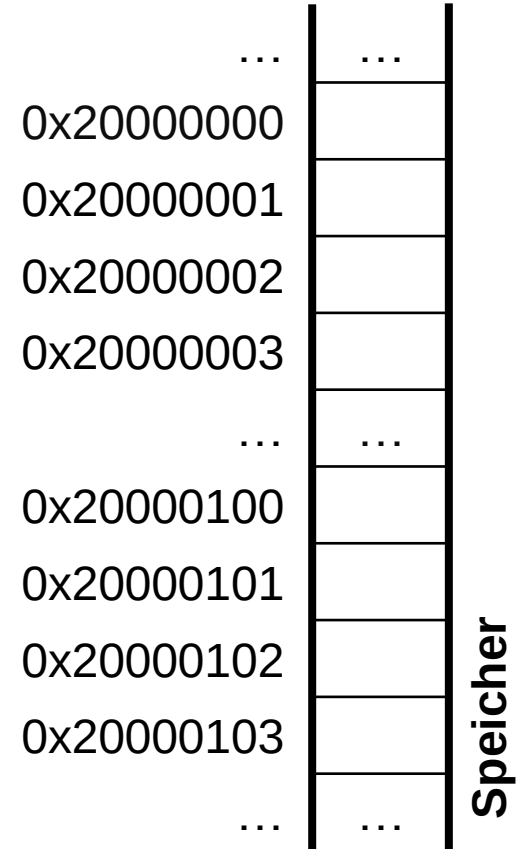
...
0x20000200
0x20000201
0x20000202
0x20000203
...



Zeiger (Beispiel 2)

Zeiger auf absolute Adresse

```
#define ADRESSE 0x20000100
...
long int *abs_ptr;
...
// Zeiger initialisieren
abs_ptr = (long int*)ADRESSE;
...
// Zugriff auf absolute Adresse
*abs_ptr = (long int)10;
```



Zeiger (Beispiel 2)

Zeiger auf absolute Adresse

```
#define ADRESSE 0x20000100
```

```
...
```

```
long int *abs_ptr;
```

```
...
```

```
// Zeiger initialisieren
```

```
abs_ptr = (long int*)ADRESSE;
```

```
...
```

```
// Zugriff auf absolute Adresse
```

```
*abs_ptr = (long int)10;
```

Zeiger-Variable
abs_ptr

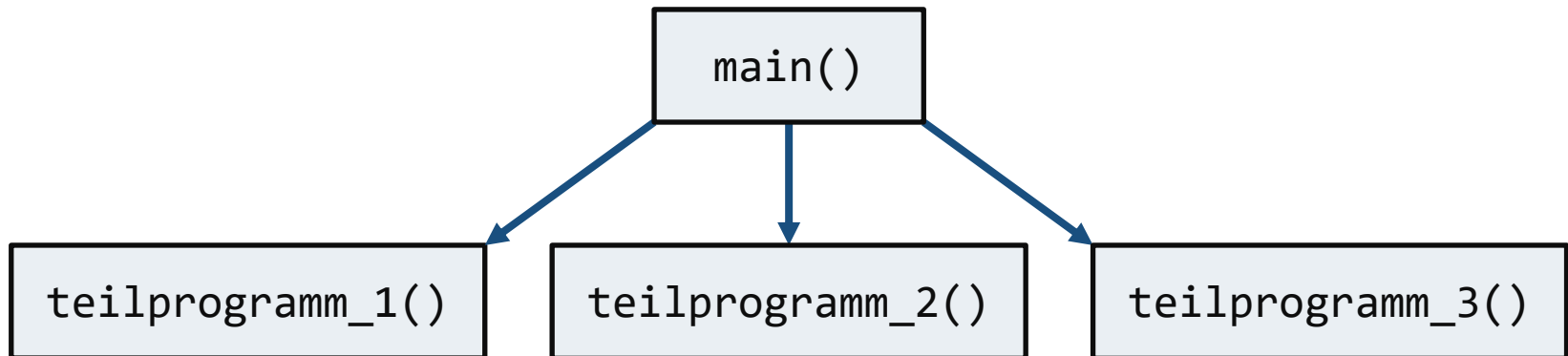
...	...
0x20000000	20
0x20000001	00
0x20000002	01
0x20000003	00
...	...
0x20000100	00
0x20000101	00
0x20000102	00
0x20000103	0A
...	...

Speicher

Funktionen (Unterprogramme)

Funktionen (Unterprogramme)

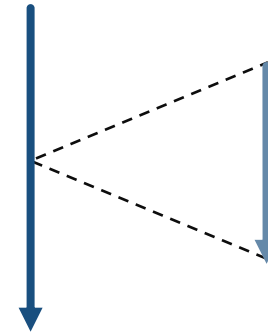
- Strukturierung des Programms
- Zusammenfassen von Teilen des Programms unter einem eigenen Namen



- jeder Sub-Algorithmus (Funktion) löst ein Teilproblem
- der Haupt-Algorithmus (main-Funktion) enthält die Ablaufsteuerung

Funktionen (Unterprogramme)

- eine Funktion wird aus einer anderen Funktion heraus aufgerufen
- eine Funktion kann nur außerhalb einer anderen Funktion definiert werden



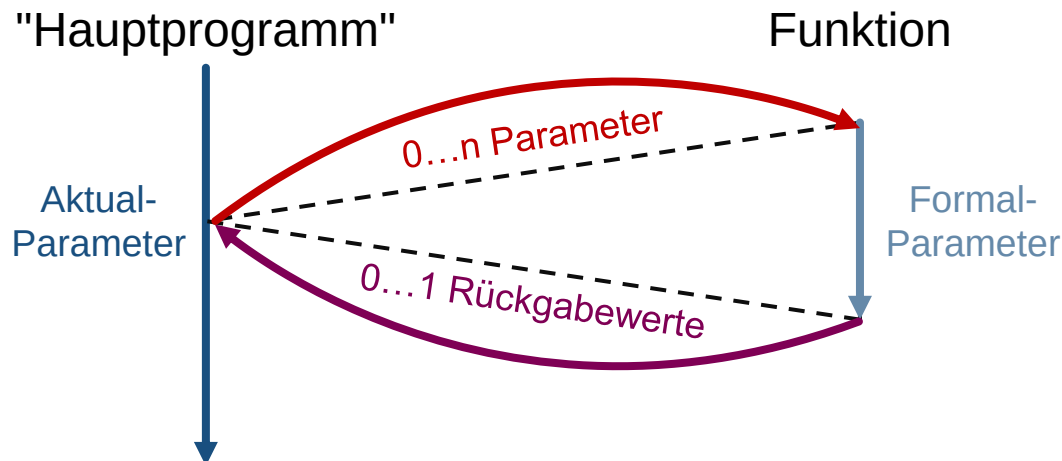
main()

teilprogramm_1()

teilprogramm_2()

Funktionen (Unterprogramme)

- Strukturierung des Programms
- Zusammenfassen von Teilen des Programms unter einem eigenen Namen
- Aufruf der Funktion aus einer anderen Funktion



Funktionen (Unterprogramme)

Funktionsaufruf:

```
variable = funktionsname(aktualparameter_1,  
                          aktualparameter_2,  
                          ...  
                          aktualparameter_n);
```

Funktionsdefinition:

```
rueckgabetyf funktionsname(typ_1 formalparameter_1,  
                           typ_2 formalparameter_2,  
                           ...  
                           typ_n formalparameter_n)  
  
{  
    ...  
    return(rueckgabewert);  
}
```

Funktionen (Unterprogramme)

Funktionsaufruf:

```
variable = funktionsname(aktualparameter_1,  
                        aktualparameter_2,  
                        ...  
                        aktualparameter_n);
```

Funktionsdefinition:

```
rueckgabetyf funktionsname(typ_1 formalparameter_1,  
                          typ_2 formalparameter_2,  
                          ...  
                          typ_n formalparameter_n)
```

Funktionskopf

```
{  
    ...  
    return(rueckgabewert);  
}
```

Funktionsrumpf

Funktionen (Unterprogramme)

Funktionsaufruf:

```
variable = funktionsname(aktualparameter_1,  
                          aktualparameter_2,  
                          ...  
                          aktualparameter_n);
```

Parameter beim
Aufruf der Funktion

Funktionsdefinition:

```
rueckgabetyf funktionsname(typ_1 formalparameter_1,  
                           typ_2 formalparameter_2,  
                           ...  
                           typ_n formalparameter_n)
```

Übernahme in die Funktion (Kopie)

```
{  
    ...  
    return(rueckgabewert);  
}
```

Programm (verwendet die Parameter)

Funktionen (Unterprogramme)

Funktionsaufruf:

```
variable = funktionsname(aktualparameter_1,  
                        aktualparameter_2,  
                        ...  
                        aktualparameter_n);
```

nur ein
Rückgabewert
möglich

Funktionsdefinition:

```
rueckgabety funktionsname(typ_1 formalparameter_1,  
                          typ_2 formalparameter_2,  
                          ...  
                          typ_n formalparameter_n)
```

Datentyp des
Rückgabewerts

```
{  
    ...  
    return(rueckgabewert);  
}
```

Funktionen (Unterprogramme)

Beispiel: Funktion ohne Parameter und ohne Rückgabewert

```
// Funktionsprototypen
```

```
void main(void);
```

```
void test(void);
```

```
...
```

```
// Hauptprogramm
```

```
void main(void)
```

```
{
```

```
...
```

```
test();
```

```
...
```

```
}
```

```
// Funktion
```

```
void test(void)
```

```
{
```

```
...
```

```
}
```

Funktionen (Unterprogramme)

Beispiel: Funktion mit Parameter und ohne Rückgabewert

```
// Funktionsprototypen  
void main(void);  
void test(int alpha);
```

...

```
// Hauptprogramm  
void main(void)  
{  
    int var_a;  
    ...  
    test(var_a);  
    ...  
}
```

```
// Funktion  
void test(int alpha)  
{  
    ...  
}
```

Parameterübergabe:

Der Wert des Aktualparameters var_a ist in der Funktion unter dem Namen des Formalparameters alpha vorhanden.

Übergabe per Value

Funktionen (Unterprogramme)

Beispiel: Funktion mit Parameter und mit Rückgabewert

```
// Funktionsprototypen
void main(void);
int test(int alpha, int beta);

...

// Hauptprogramm
void main(void)
{
    int var_a;
    int var_b;
    int var_x;
    ...
    var_x = test(var_a, var_b);
    ...
}
```

```
// Funktion
int test(int alpha, int beta)
{
    int var_y;
    ...
    return(var_y);
}
```

Parameterübergabe:

Die Werte der Aktualparameter var_a und var_b sind in der Funktion unter den Namen der Formalparameter alpha und beta vorhanden.

Übergabe per Value

Funktionen (Unterprogramme)

Beispiel: Funktion mit Parameter und mit Rückgabewert

```
// Funktionsprototypen
void main(void);
int test(int alpha, int beta);

...

// Hauptprogramm
void main(void)
{
    int var_a;
    int var_b;
    int var_x;
    ...
    var_x = test(var_a, var_b);
    ...
}
```

```
// Funktion
int test(int alpha, int beta)
{
    int var_y;
    ...
    return(var_y);
}
```

Rückgabewert:

Der Wert der lokalen Variable `var_y` wird zurückgegeben und im aufrufenden Programm der Variablen `var_x` zugewiesen.

Funktionen (Unterprogramme)

Beispiel: Funktion mit mehreren Rückgabewerten

```
// Funktionsprototypen
void main(void);
int test(int *alpha_ptr,
        int *beta_ptr);
...

// Hauptprogramm
void main(void)
{
    int var_a;
    int var_b;
    int var_x;
    ...
    var_x = test(&var_a, &var_b);
    ...
}
```

```
// Funktion
int test(int *alpha_ptr,
        int *beta_ptr)
{
    int var_y;
    ...
    *alpha_ptr = ...;
    ...
    return(var_y);
}
```

Datenflussrichtung nur vom aufrufenden Programm zum aufgerufenen Programm

Funktionen (Unterprogramme)

Beispiel: Parameterübergabe per Referenz

// Hauptprogramm

void main(**void**)

{
 int error;
 short int var_x;

lokale Variable
von main: var_x

error = func1(&var_x);

...

Adresse von var_x.
Die Zahl 0x20000000
wird an die Funktion
übergeben.

// Funktion

int func1(**short int** *x_ptr)

{

*x_ptr = 10;

...

Schreiben auf diese Adresse
bedeutet: Inhalt der Variable
var_x verändern.

In x_ptr steht eine
Kopie von &var_x (die
Zahl 0x20000000),
d.h. der Pointer zeigt
auf die Adresse
0x20000000 im
Speicher.

...	...
0x20000000	00
0x20000001	0A
0x20000002	
0x20000003	
...	
0x20000100	
0x20000101	
0x20000102	
0x20000103	
...	...

Speicher

Variablen

Variablen

Parameterübergabe, lokale Variablen → **Stack**

```
// Funktion
```

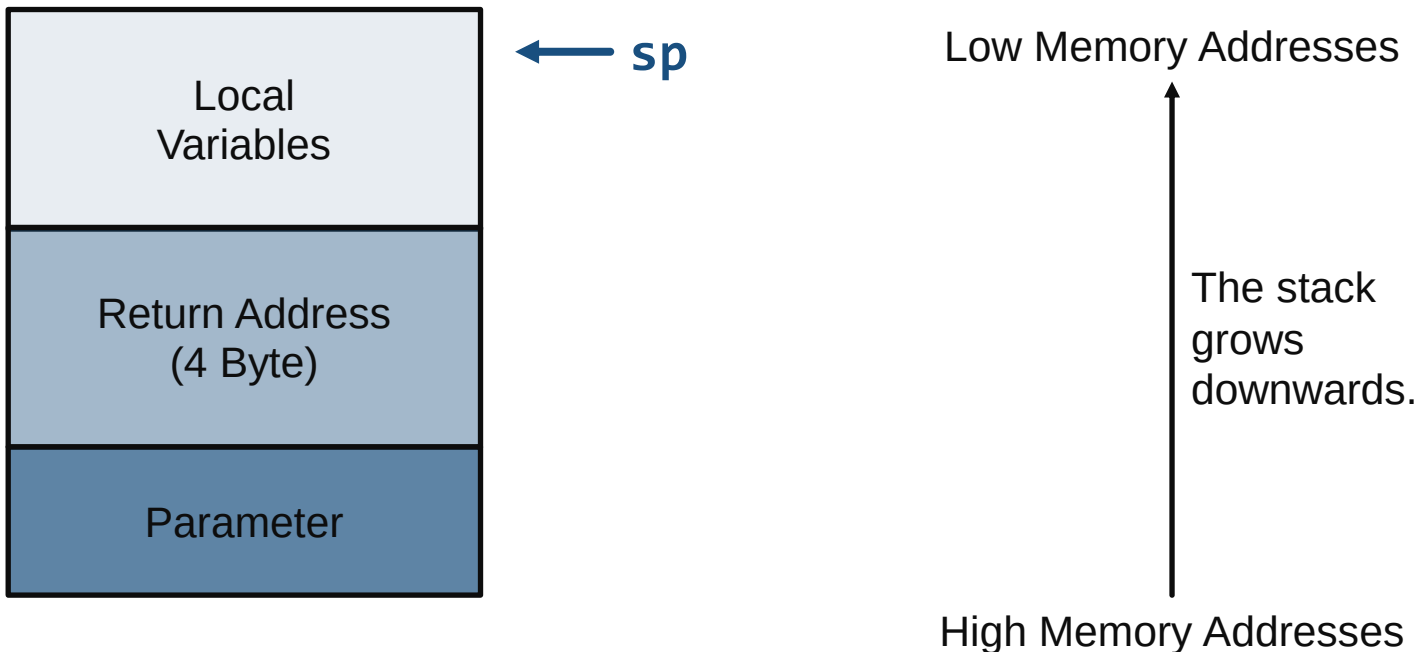
```
int func1(int count, char *buffer_ptr, float analog_in)
{
    int loop;
    float zahl;
    int error;

    ...

    return(error);
}
```

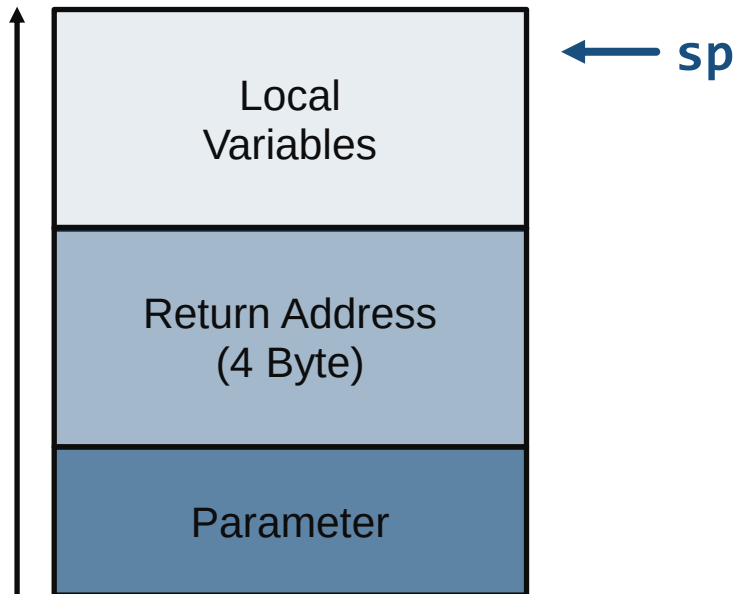
Variablen

Der C-Compiler erzeugt beim Aufruf einer Funktion das Stack-Layout



Variablen

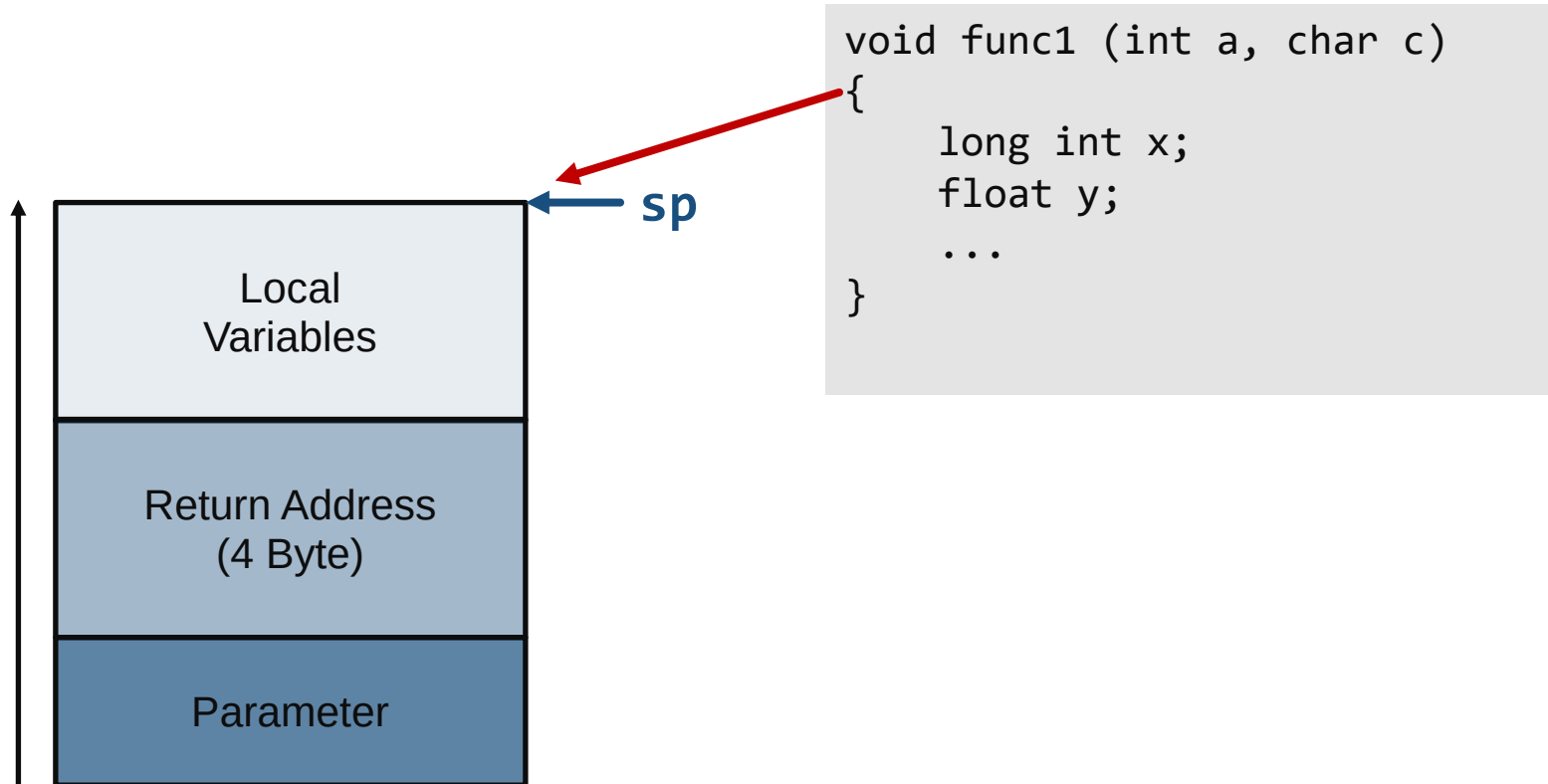
Der C-Compiler erzeugt beim Aufruf einer Funktion das Stack-Layout



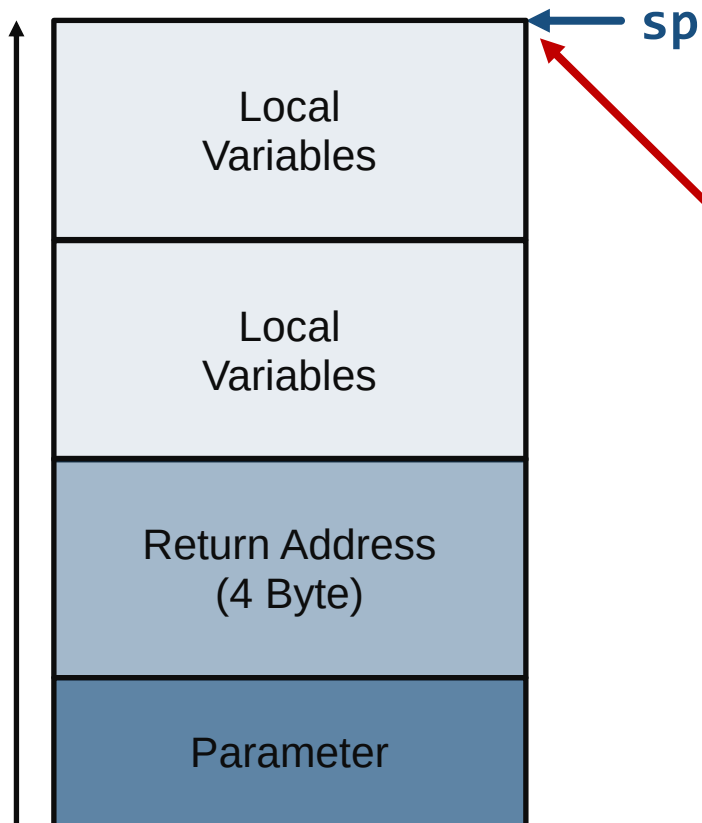
Zugriff auf lokale Variablen

Basisadresse: Stackpointer
Position der Variablen: konstanter Offset

Variablen



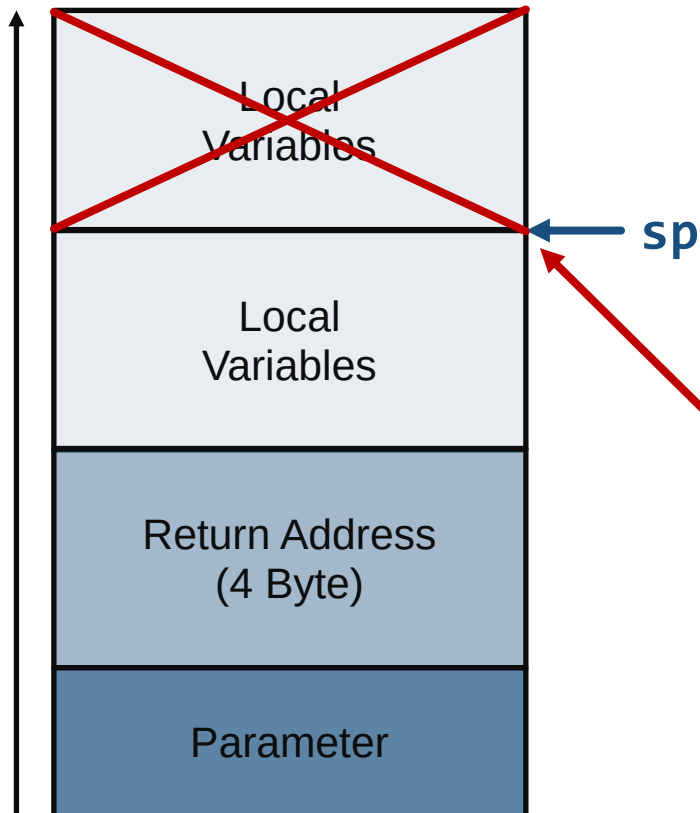
Variablen



```
void func1 (int a, char c)
{
    long int x;
    float y;
    ...

    for(...)
    {
        int n;
        char m;
        ...
    }
    ...
}
```


Variablen



```
void func1 (int a, char c)
{
    long int x;
    float y;
    ...

    for(...)
    {
        int n;
        char m;
        ...
    }
    ...
}
```

**nur am
Blockanfang**

Speicherort und Speicherklassen

Beeinflussung des Speicherorts durch Speicherklassen

Modulvariablen

Speicherbereich: Data / Variablen

Lokale Variablen

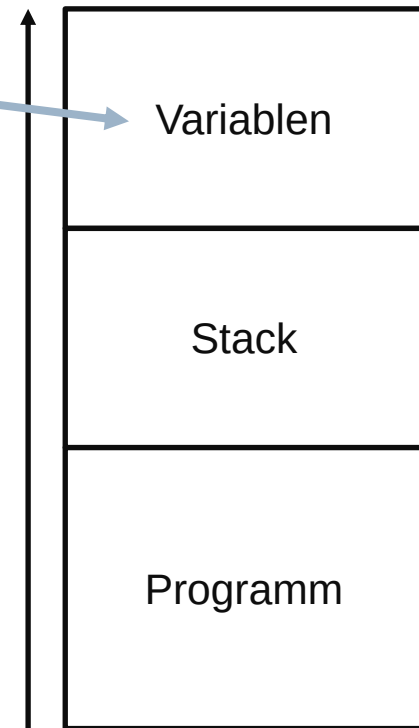
Speicherbereich: Data / Stack

Lokale Variablen, Speicherklasse static

Speicherbereich: Data / Variablen

Lokale Variablen, Speicherklasse register

Speicherbereich: Data / Stack oder
Prozessor-Register



Speicherort und Speicherklassen

Beeinflussung des Speicherorts durch Speicherklassen

Modulvariablen

Speicherbereich: Data / Variablen

Lokale Variablen

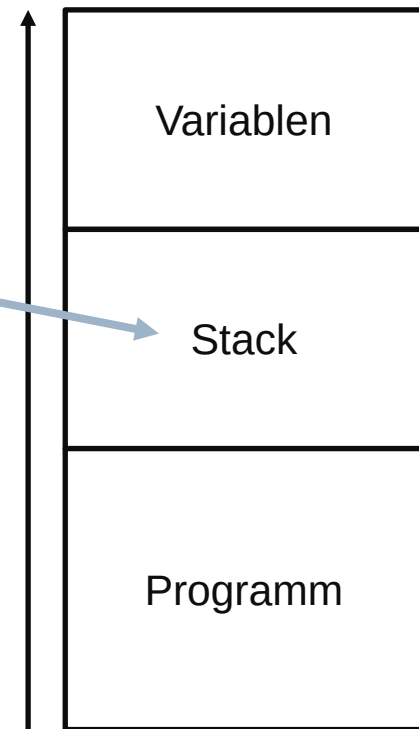
Speicherbereich: Data / Stack

Lokale Variablen, Speicherklasse `static`

Speicherbereich: Data / Variablen

Lokale Variablen, Speicherklasse `register`

Speicherbereich: Data / Stack oder
Prozessor-Register



Speicherort und Speicherklassen

Beeinflussung des Speicherorts durch Speicherklassen

Modulvariablen

Speicherbereich: Data / Variablen

Lokale Variablen

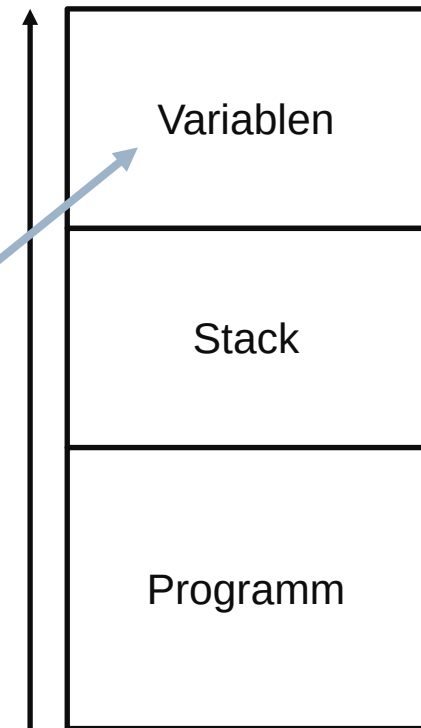
Speicherbereich: Data / Stack

Lokale Variablen, Speicherklasse **static**

Speicherbereich: Data / Variablen

Lokale Variablen, Speicherklasse **register**

Speicherbereich: Data / Stack oder
Prozessor-Register



Speicherort und Speicherklassen

Beeinflussung des Speicherorts durch Speicherklassen

Modulvariablen

Speicherbereich: Data / Variablen

Lokale Variablen

Speicherbereich: Data / Stack

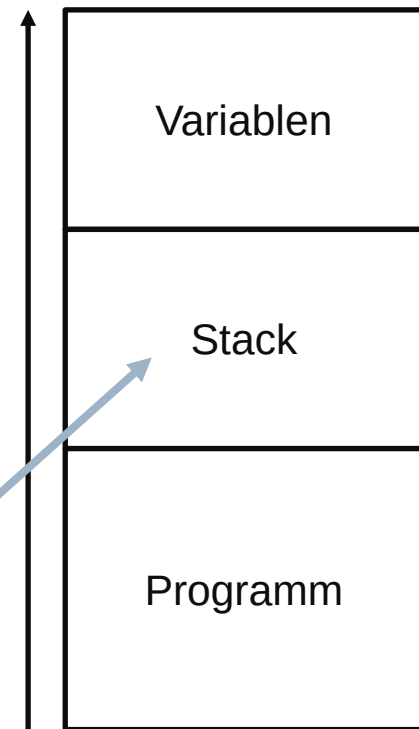
Lokale Variablen, Speicherklasse `static`

Speicherbereich: Data / Variablen

Lokale Variablen, Speicherklasse `register`

Speicherbereich: Data / Stack oder
Prozessor-Register

CPU Register



Lebensdauer von Variablen

```
#include ...

int sum;
...
void fkt1(void)
{
    int count;
    ...
    {
        int i;
        ...
    }
    ...
}

void fkt2(void)
{
    static char state;
    ...
}

void main(void)
{
    ...
}
```

Lebensdauer von Variablen

```
#include ...
```

```
int sum;
```

```
...
```

```
void fkt1(void)
```

```
{
```

```
    int count;
```

```
    ...
```

```
{
```

```
    int i;
```

```
    ...
```

```
}
```

```
...
```

```
}
```

```
void fkt2(void)
```

```
{
```

```
    static char state;
```

```
    ...
```

```
}
```

```
void main(void)
```

```
{
```

```
    ...
```

```
}
```

Modulvariable: wird beim Starten des Programms angelegt und lebt solange wie das Programm (main) einschließlich der aufgerufenen Funktionen

Lebensdauer von Variablen

```
#include ...
```

```
int sum;
...
void fkt1(void)
{
    int count;
    ...
    {
        int i;
        ...
    }
    ...
}

void fkt2(void)
{
    static char state;
    ...
}

void main(void)
{
    ...
}
```

Modulvariable: wird beim Starten des Programms angelegt und lebt solange wie das Programm (main) einschließlich der aufgerufenen Funktionen

Lebensdauer von Variablen

```
#include ...  
  
int sum;  
...  
void fkt1(void)
```

```
{  
    int count;  
    ...  
    {  
        int i;  
        ...  
    }  
    ...  
}
```

Lokale Variable: wird beim Betreten des Blockes angelegt und verschwindet beim Verlassen des Blockes



```
void fkt2(void)  
{  
    static char state;  
    ...  
}
```

```
void main(void)  
{  
    ...  
}
```

Lebensdauer von Variablen

```
#include ...

int sum;
...
void fkt1(void)
{
    int count;
    ...
    {
        int i;
        ...
    }
    ...
}

void fkt2(void)
{
    static char state;
    ...
}

void main(void)
{
    ...
}
```

Lokale Variable: wird beim Betreten des Blockes angelegt und verschwindet beim Verlassen des Blockes



Lebensdauer von Variablen

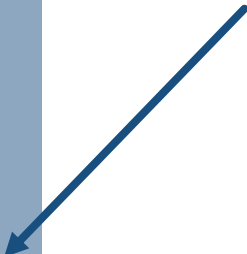
```
#include ...
```

```
int sum;
...
void fkt1(void)
{
    int count;
    ...
    {
        int i;
        ...
    }
    ...
}

void fkt2(void)
{
    static char state;
    ...
}

void main(void)
{
    ...
}
```

Lokale Variable: wird beim Starten des Programms im Speicherbereich der Modulvariablen / globalen Variablen angelegt und wird dadurch permanent, d.h. behält den zugewiesenen Wert auch zwischen zwei Funktionsaufrufen bei. Die Variable ist nur innerhalb des Blocks sichtbar.



Lebensdauer von Variablen

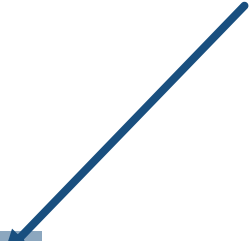
```
#include ...

int sum;
...
void fkt1(void)
{
    int count;
    ...
    {
        int i;
        ...
    }
    ...
}

void fkt2(void)
{
    static char state;
    ...
}

void main(void)
{
    ...
}
```

Lokale Variable: wird beim Starten des Programms im Speicherbereich der Modulvariablen / globalen Variablen angelegt und wird dadurch permanent, d.h. behält den zugewiesenen Wert auch zwischen zwei Funktionsaufrufen bei. Die Variable ist nur innerhalb des Blocks sichtbar.

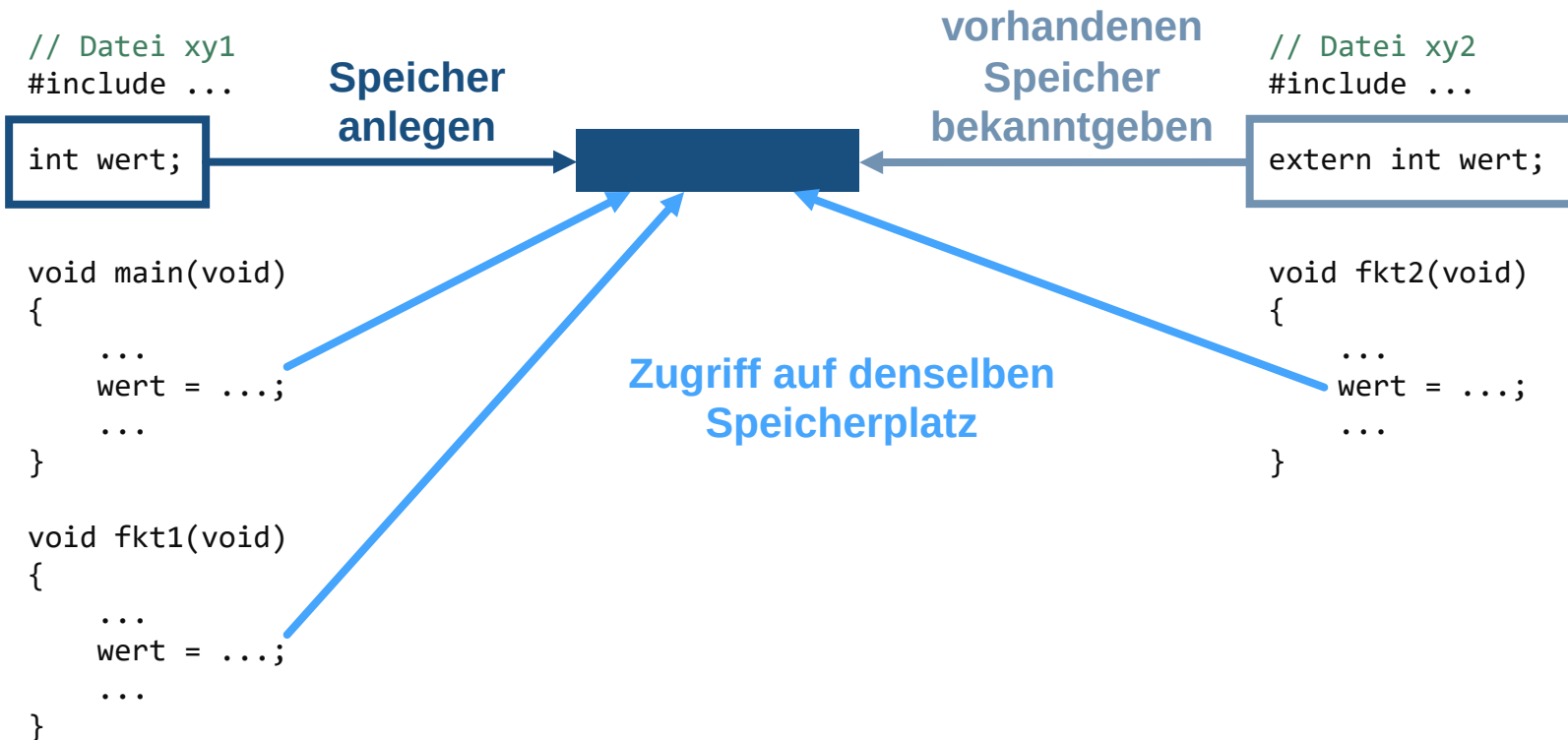


Sichtbarkeit

Lebensdauer von Variablen

Speicherklasse extern

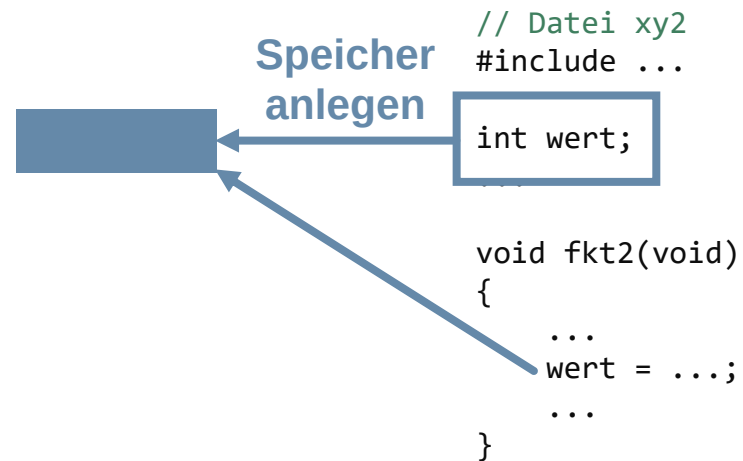
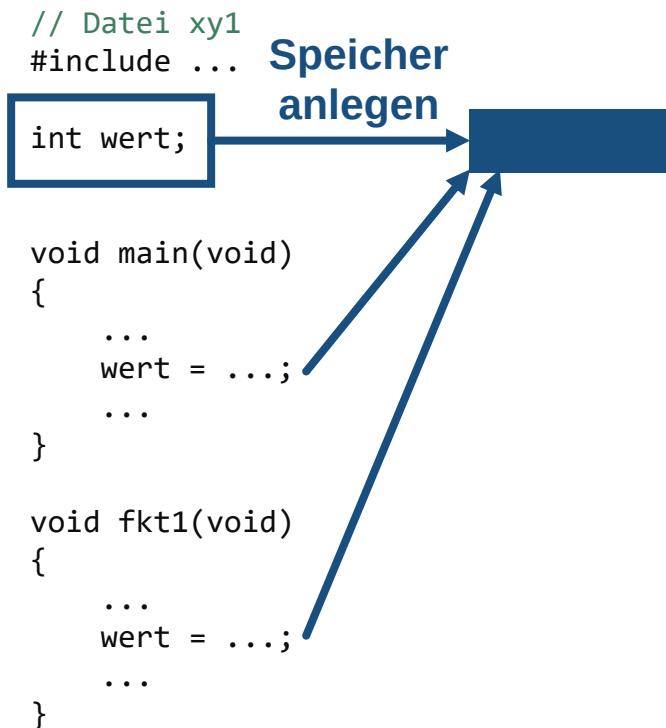
Modulvariable, Speicherklasse extern $\longrightarrow$ Globale Variable



Lebensdauer von Variablen

Speicherklasse extern

Gegenbeispiel



Bibliotheksfunktion scanf

Bibliotheksfunktion scanf

```
scanf("...", v1, v2, ...);
```

Platzhalter für
Variableninhalte

Liste von
Variablen

Beispiel:

```
int count;
```

```
scanf("%i", &count);
```

Platzhalter für die
Variable

Werte **lesen** über Parameterleiste:
Parameterübergabe per Reference

Formatelemente scanf

Definition von Character-Feldern (String-Variablen)

```
char inf2_string[10];
```

Eingabe-Formatelement für Character-Feldern (String-Variablen)

`%(b)s` char-array



optional: Maximalbreite des Eingabefeldes

Beispiel:

```
scanf("%s", &inf2_string[0]);      // Tastatureingabe z.B. Test
```

Formatelemente scanf

Definition von Integer-Variablen

```
int var1;
```

```
short int var2;
```

```
long int var3;
```

Eingabe-Formatelement für Integer-Variablen

`%(b)d` int, dezimale Darstellung

`%(b)ld` long int, dezimale Darstellung

`%(b)hd` short int, dezimale Darstellung

`%(b)x` int, hexadezimale Darstellung

`%(b)lx` long int, hexadezimale Darstellung

`%(b)hx` short int, hexadezimale Darstellung

 **optional: Maximalbreite des Eingabefeldes**

Formatelemente scanf

Definition von Integer-Variablen

```
int var1;
```

```
short int var2;
```

```
long int var3;
```

Beispiel:

```
scanf("%d", &var1);           // Tastatureingabe z.B. 18  
scanf("%5hd", &var2);         // Tastatureingabe z.B. 12345678  
scanf("%1x", &var3);         // Tastatureingabe z.B. 7FFFFFFF
```

Formatelemente scanf

Definition von Float-Variablen (Gleitkommazahlen)

```
float inf2_f1;
```

Eingabe-Formatelement für Float-Variablen

`%(b)f` float-Variable (Darstellung: ddd.ddddd)

 optional: Maximalbreite des Eingabefeldes

Beispiele:

```
scanf("%f", &inf2_f1);           // Tastatureingabe z.B. 3.14
```

```
scanf("%f", &inf2_f1);           // Tastatureingabe z.B. 3.
```

Formatelemente scanf

```
void main(void)
{
    int loop;
    short int zahl1;
    long int zahl2[20];
    ...

    for(loop = 0; loop < 20; loop++)
    {
        printf("\n\r zahl1 eingeben: ")
        scanf("%d", &zahl1);
        ...
        printf("\n\r zahl2 eingeben: ")
        scanf("%ld", &zahl2[loop]);
        ...
    }
    ...
}
```